

Midterm Lab Exam — Submission Form

Student Information System with GitHub Integration

Field	Answer
GitHub Repository URL	<code>https://github.com/FundanoJoseph/Midterm-Lab-Exam</code>
Project Folder	<code>student-info-system/</code>
Pull Request	<code>arena/8a623f42-midterm-lab-exam</code> → <code>main</code>
Student Name	Joseph Fundano
Course / School	BS Information Technology, Pan Pacific University
Date Submitted	October 8, 2026

1. Features Implemented

Student Data Management (30 pts)

- ✓ **Add new students** — validated input, duplicate-ID protection
- ✓ **View student records** — formatted table of all students
- ✓ **Update student information** — partial updates (leave a field blank to keep it)
- ✓ **Delete students** — with confirmation prompt
- ✓ **JSON/XML data storage** — records stored in `data/students.json` (loaded on start, saved atomically on every change) and exportable to XML as well as CSV

Cloud-Ready Architecture (25 pts)

- ✓ **Modular code structure** — `models/`, `services/`, `utils/` layers with clear separation of concerns
- ✓ **Configuration management** — `config/config.json` plus `SIS_DATA_FILE` / `SIS_LOG_LEVEL` environment overrides
- ✓ **Error handling** — custom exceptions (`StudentNotFoundError`, `DuplicateStudentError`, `ValidationError`), corrupt-file recovery, atomic writes
- ✓ **Logging system** — rotating file logger (`logs/app.log`, 1 MB × 3 backups) + console handler for errors only

GitHub Integration (25 pts)

- ✓ **Proper repository structure** — `src/`, `data/`, `config/`, `logs/`, `tests/`, `docs/`, `README`, `requirements`, `.gitignore`
- ✓ **Meaningful commit history** — 10+ logical commits with conventional messages (`feat:`, `test:`, `docs:`, `chore:`)
- ✓ **Branching strategy** — work done on feature branch `arena/8a623f42-midterm-lab-exam`, merged to `main` via pull request; suggested feature-branch workflow in `docs/git_commands.sh`
- ✓ **GitHub features utilization** — pull request for review, GitHub Actions CI (`.github/workflows/ci.yml`) that runs `pytest` and a CLI smoke test on every push/PR
- ✓ **README documentation** — setup, configuration, architecture, workflow and rubric-mapped

feature list

Code Quality (20 pts)

- ✓ Clean, readable, PEP 8-consistent code
- ✓ Docstrings on every module, class and function + inline comments
- ✓ Consistent, meaningful naming conventions

Bonus Features (+10 pts)

- ✓ **Unit tests** — 10 pytest tests covering CRUD, validation, persistence, search, CSV/XML export and corrupt-file recovery (all passing)
 - ✓ **Data validation** — student ID format, name, age range (5-100), course, email regex
 - ✓ **Search functionality** — case-insensitive search across ID, name, course and email
 - ✓ **Data export** — one-key CSV **and XML** export to data/exports/ (covers the JSON/XML storage requirement)
 - ✓ **Advanced error recovery** — corrupt JSON is backed up to `students.json.corrupt` and replaced automatically; atomic saves prevent truncated files
-

2. Challenges Faced

1. **PEP 668 externally-managed environment** — `pip install pytest` was blocked by the OS. Solved by creating a virtual environment for testing and keeping the application itself 100% standard-library so it runs anywhere with Python 3.8+.
 2. **Preventing data loss on crash** — a plain `json.dump()` can leave a truncated/corrupt file if the process dies mid-write. Solved with atomic saves: write to a temp file in the same folder, then `os.replace()`.
 3. **Recovering from a corrupt data file** — a hand-edited or partially-written `students.json` would crash the app on startup. Solved by catching decode errors, backing the bad file up to `students.json.corrupt`, logging the error, and starting with a fresh store.
 4. **Keeping the interactive menu clean** — logging to stdout interfered with the menu UI. Solved by routing only ERROR-and-above to the console while everything goes to the rotating log file.
 5. **Duplicate log handlers** — re-initializing the logger (e.g. in tests) appended duplicate handlers. Solved with a guard in `setup_logger()`.
 6. **GitHub workflow setup** — remembering to remove the starter ZIP artifact, keep `logs/` and `data/exports/` out of version control via `.gitignore`, and structure commits logically so the history reads as a clean progression.
 7. **Covering the "JSON/XML" storage requirement** — the base implementation only stored JSON, so an XML export was added using the standard-library `xml.etree.ElementTree` (with a well-formedness test), plus a new menu option and CI smoke-test coverage, without adding any third-party dependency.
-

3. How to Run

```
git clone https://github.com/FundanoJoseph/Midterm-Lab-Exam.git
cd Midterm-Lab-Exam/student-info-system
python -m src.main          # interactive menu
pytest                     # run the unit tests (after: pip install -r requirements.txt)
```

Menu options: 1. Add student · 2. View all students · 3. Update student ·
4. Delete student · 5. Search students · 6. Export to CSV · 7. Export to XML · 0. Exit